
Projet Atlas

Portail Helpdesk GLPI — Plan de Reprise d'Activité

Procédure d'exploitation du POC — déroulé pas à pas

Contraintes de reprise : $RPO \leq 20 \text{ min}$ | $RTO \leq 40 \text{ min}$

Méthode : 100% IaC (Ansible) — aucun « clic-clic » non tracé

Hyperviseur : Proxmox VE 8.4 (pve01, 138.201.135.108)

Périmètre : 9 conteneurs LXC — 3 VLAN (DMZ / LAN-ADMIN / MGMT)

Table des matières

1	Présentation du POC	2
1.1	Objectif	2
1.2	Architecture	2
1.3	Déroulé de la procédure (ordre chronologique)	2
2	Prérequis et poste de pilotage	2
3	Étape 1 — Accéder à chaque application et outil	3
3.1	1.a — GLPI (le portail de tickets)	3
3.2	1.b — Zabbix (la supervision)	3
3.3	1.c — Grafana (les tableaux de bord)	3
4	Étape 2 — Déployer / redéployer l'infrastructure	4
4.1	2.a — Filet 1 : vérification de syntaxe	4
4.2	2.b — Filet 2 : vérifier le périmètre	4
4.3	2.c — Filet 3 : simulation (dry-run) avec diff	4
4.4	2.d — Application réelle	5
5	Étape 3 — Éprouver la résilience (chaos + supervision)	5
6	Étape 4 — Vérifier les sauvegardes (RPO)	6
7	Étape 5 — Reprise après sinistre : restauration complète	6
8	Étape 6 — Reprise ciblée : restauration granulaire	7
9	Aide-mémoire — commandes essentielles	8
10	Annexe — Corrections intégrées à l'IaC (juillet 2026)	8

1. Présentation du POC

1.1 Objectif

Atlas est un **portail de tickets d'assistance (GLPI)** déployé de bout en bout par Infrastructure as Code. L'objectif du POC est de démontrer qu'un incident majeur (perte de VM, corruption de données, panne de service) peut être détecté puis corrigé **dans les délais contractuels** : $RPO \leq 20$ minutes (perte de données maximale) et $RTO \leq 40$ minutes (durée de remise en service).

1.2 Architecture

Deux chemins réseau *indépendants* coexistent :

- **Chemin utilisateur (web)** : Internet → Reverse Proxy TLS (atlrp01p, DMZ) → GLPI (atlapp01p) → MariaDB (atldb01p).
- **Chemin administration (SSH)** : Admin → Bastion (atlbst01p, MGMT) → rebond vers tous les LXC. Aucun service n'est exposé sur Internet.

Conteneur	VMID	IP	Rôle
atlbst01p	200	10.90.0.10	Bastion SSH — point d'entrée admin unique
atlrp01p	201	10.20.0.10	Reverse proxy Nginx (TLS)
atlapp01p	202	10.20.0.11	Application GLPI (PHP-FPM)
atldb01p	203	10.30.0.10	Base de données MariaDB
atlsmt01p	204	10.30.0.11	Relais SMTP Postfix
atlbxb01p	205	10.30.0.12	Supervision Zabbix (serveur + frontend)
atlbkp01p	206	10.30.0.13	Serveur de sauvegarde (copies distantes)
atlgf01p	207	10.30.0.14	Grafana (visualisation)
atltst01p	208	10.20.0.20	Machine de test (chaos engineering)

TABLE 1 – Les 9 conteneurs LXC du POC Atlas.

1.3 Déroulé de la procédure (ordre chronologique)

Ce document se lit **dans l'ordre**, comme on déroulerait une démonstration du POC :

- Étape 1. Accéder à chaque application et outil** — on ouvre GLPI, Zabbix puis Grafana et on vérifie qu'ils répondent (le résultat visible).
- Étape 2. Déployer / redéployer l'infrastructure** — comment tout est produit par Ansible, et comment le rejouer *sans rien casser* (tests d'abord).
- Étape 3. Éprouver la résilience** — on provoque un incident (chaos) et on vérifie que Zabbix le détecte, puis on répare.
- Étape 4. Vérifier les sauvegardes** — la chaîne qui garantit le RPO.
- Étape 5. Reprise après sinistre : restauration complète** — perte de VM (RTO).
- Étape 6. Reprise ciblée : restauration granulaire** — quelques tickets perdus.

2. Prérequis et poste de pilotage

Avant de commencer, on se place sur le **control node** : l'hyperviseur **pve01** (138.201.135.108), où réside le dépôt Ansible.

```
# Se connecter au control node
ssh root@138.201.135.108
cd /root/atlas          # repertoire du depot Ansible

# Verifier les outils et l'inventaire AVANT toute action
ansible --version
ansible-inventory -i develop/hosts.yaml --graph  # groupes :
    proxmox/management/dmz/lan_admin
```

Astuce

`ansible-inventory -graph` confirme que l'inventaire est lisible et que tous les hôtes attendus sont là — c'est le tout premier filet de sécurité, avant même d'ouvrir une application.

Éléments sensibles (hors dépôt Git). Clé SSH `/root/.ssh/atlas_ed25519` (rebond bastion); coffre `.vault_pass` (déchiffre les secrets `vault_*`); `ansible.cfg` porte déjà l'inventaire, le `roles_path` et le vault.

Attention

Reprise du control node. Si `pve01` est lui-même sinistré, Ansible doit tourner depuis un poste de secours : copier `atlas_ed25519` + `.vault_pass` (coffre hors site) puis `ansible-playbook ... -e ansible_ssh_private_key_file=<chemin>`. Bascule à tester régulièrement pour tenir le RTO.

3. Étape 1 — Accéder à chaque application et outil

On commence par le résultat visible : les services tournent-ils ?

Aucun service n'est exposé sur Internet. On y accède par **tunnel SSH** depuis son poste : on ouvre le tunnel, puis on pointe le navigateur sur `localhost`. On visite les trois interfaces l'une après l'autre.

3.1 1.a — GLPI (le portail de tickets)

```
# Ouvrir le tunnel (via le reverse proxy TLS), puis garder le terminal ouvert
ssh -L 8443:10.20.0.10:443 root@138.201.135.108
```

Naviguer sur <https://localhost:8443> (certificat interne auto-signé : « Avancé → Continuer »). Identifiants : voir README / vault. **À vérifier** : la page de connexion s'affiche, on se connecte, la liste des tickets est présente.

3.2 1.b — Zabbix (la supervision)

```
ssh -L 8080:10.30.0.12:8080 root@138.201.135.108
```

Naviguer sur <http://localhost:8080>. **À vérifier** : *Monitoring* → *Hosts* affiche les **10 hôtes en vert** (Available), et *Monitoring* → *Problems* est vide. C'est l'état nominal de référence auquel on comparera après l'incident (Étape 3).

3.3 1.c — Grafana (les tableaux de bord)

```
ssh -L 3000:10.30.0.14:3000 root@138.201.135.108
```

Naviguer sur <http://localhost:3000>. **À vérifier** : le tableau de bord « Atlas — Vue d'ensemble » remonte bien les métriques (Grafana lit Zabbix via un compte en lecture seule).

Pourquoi cette étape ?

On visite les applis *en premier* pour établir la « photo » de l'état sain : services web up, 10 hôtes verts, dashboards alimentés. Sans ce point de référence, on ne pourrait pas démontrer plus loin qu'un incident est bien détecté puis résorbé.

Outil	Tunnel	URL locale
GLPI	ssh -L 8443:10.20.0.10:443 ...	https://localhost:8443
Zabbix	ssh -L 8080:10.30.0.12:8080 ...	http://localhost:8080
Grafana	ssh -L 3000:10.30.0.14:3000 ...	http://localhost:3000

TABLE 2 – Récapitulatif des accès (à ouvrir dans cet ordre).

4. Étape 2 — Déployer / redéployer l'infrastructure

Les services répondent. Voyons comment ils sont produits — et comment rejouer le playbook sans rien casser.

Tout l'environnement de l'Étape 1 est généré par Ansible (`site.yaml`). On peut le rejouer intégralement ou par morceau. **Règle d'or** : jamais d'exécution en production sans passer d'abord les trois filets ci-dessous, *dans l'ordre*.

4.1 2.a — Filet 1 : vérification de syntaxe

```
ansible-playbook site.yaml --syntax-check
```

Pourquoi cette étape ?

Analyse le YAML et le graphe des rôles *sans se connecter* à aucune machine. Détecte les fautes d'indentation et les rôles introuvables. Coût nul, à lancer après chaque édition.

4.2 2.b — Filet 2 : vérifier le périmètre

```
ansible-playbook site.yaml --list-hosts      # sur quels hotes ca va tourner
ansible-playbook site.yaml --list-tasks      # quelles taches, dans quel ordre
```

Pourquoi cette étape ?

On confirme *quoi* et *où* avant d'agir : ne jamais toucher un hôte non prévu. `-list-tasks` montre aussi où intervient chaque tag.

4.3 2.c — Filet 3 : simulation (dry-run) avec diff

```
ansible-playbook site.yaml --check --diff      # simulation
globale
```

```
ansible-playbook site.yaml --check --diff --tags zabbix --limit atlzbx01p # ciblee
```

Pourquoi cette étape ?

`-check` exécute en « mode blanc » (aucune écriture) et `-diff` affiche les changements ligne à ligne. C'est la répétition générale : si le diff surprend, on corrige *avant* d'appliquer.

Attention

Le mode `-check` n'est pas parfait : les tâches dépendant d'un résultat précédent (API, `shell`) peuvent être signalées à tort. Il valide l'intention, pas un résultat garanti.

4.4 2.d — Application réelle

```
ansible-playbook site.yaml # deployment complet
(idempotent)
ansible-playbook site.yaml --tags zabbix # supervision seule
ansible-playbook site.yaml --tags proxmox_network # firewall/VLAN de
l'hoste
ansible-playbook site.yaml --tags zabbix --limit atlzbx01p # 1 role + 1 hôte
```

Pourquoi cette étape ?

Le playbook est **idempotent** : le rejouer sur une infra conforme ne change rien (« ok », zéro « changed »). Les **tags** et `-limit` appliquent le principe de moindre impact — on ne redéploie que ce qui est nécessaire.

Tag	Périmètre
proxmox_network	Réseau VLAN + firewall iptables de l'hyperviseur
proxmox_provision	Création/configuration des conteneurs LXC
common	Socle commun (durcissement SSH, UFW, NTP)
bastion	Bastion SSH
mariadb	Base de données
backup	Chaîne de sauvegarde
glpi / glpi_seed	Application GLPI / données de démo
reverse_proxy	Reverse proxy Nginx TLS
zabbix	Serveur + agents de supervision
grafana	Tableaux de bord
testlab	Machine de chaos engineering

TABLE 3 – Tags disponibles dans `site.yaml`.

5. Étape 3 — Éprouver la résilience (chaos + supervision)

L'infra est déployée et supervisée. On casse volontairement quelque chose pour prouver que Zabbix le voit.

La machine de lab `atlzbx01p` embarque `chaos.sh` : un générateur d'incidents **contrôlés et réversibles**. On déclenche une panne, on retourne dans Zabbix (Étape 1.b) constater l'alerte, puis on répare.

```
# 1) Provoquer un incident sur atlzbx01p (depuis le control node, en une ligne)
```

```

ansible atl1st01p -m command -a "/opt/atlas/scripts/chaos.sh service" # arret de
    nginx
# variantes : cpu (saturation CPU) | disk (remplissage) | run (scenario aleatoire)

# 2) Observer : dans Zabbix (http://localhost:8080), un probleme apparait sur
    atl1st01p

# 3) Reparer : retour a l'etat nominal
ansible atl1st01p -m command -a "/opt/atlas/scripts/chaos.sh clean"

```

Pourquoi cette étape ?

Déclencher un incident *connu* valide la boucle complète « je casse → ça alerte → je répare ». C'est la preuve que la supervision (mise en place à l'Étape 2) fonctionne réellement, et pas seulement sur le papier.

Attention

`chaos.sh` contient un **garde-fou** : il refuse de s'exécuter ailleurs que sur l'hôte de lab attendu (`hostname -s`), pour ne jamais arrêter Nginx sur le vrai reverse proxy. Un cron déclenche aussi un incident toutes les 2 h, réparé automatiquement 45 min après.

6. Étape 4 — Vérifier les sauvegardes (RPO)

Avant de simuler une catastrophe, on s'assure que le filet de données existe et qu'il est frais.

Les dumps GLPI sont produits **toutes les 15 minutes** (cron 5,20,35,50 * * * *) : RPO ≤ 15 min, mieux que la cible de 20 min. Chaque dump est vérifié (`gzip -t`) puis répliqué hors-site sur `atl1b1p01p`.

```

# Derniers dumps locaux (sur la BDD atl1db01p)
ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'

# Replication hors-site (sur le backup atl1b1p01p)
ssh root@10.30.0.13 'ls -lht /var/backups/atlas/db-remote/ | head'

# Journal de dump
ssh root@10.30.0.10 'tail -5 /var/log/atlas/dump_glpi.log'

```

Pourquoi cette étape ?

Confirmer que le dump le plus récent est présent *des deux côtés* (local + hors-site) et daté de moins de 15 minutes prouve que la chaîne de sauvegarde est vivante — c'est la condition sine qua non pour tenir le RPO lors des restaurations des Étapes 5 et 6.

7. Étape 5 — Reprise après sinistre : restauration complète

Scénario catastrophe : un ou plusieurs conteneurs sont perdus. On reconstruit tout et on mesure le RTO.

`pra_restore_full.yaml` reprovisionne les LXC, reconstruit les services dans l'ordre de dépendance, restaure les données (dernier dump automatiquement), puis **mesure et journalise** le RTO.

```
# 1) Tests prealables (memes filelets qu'a l'Etape 2)
ansible-playbook pra_restore_full.yaml --syntax-check
ansible-playbook pra_restore_full.yaml --list-tasks

# 2) Restauration complete (dernier dump distant utilise automatiquement)
ansible-playbook pra_restore_full.yaml

# 3) Restauration a partir d'un dump precis (optionnel)
ansible-playbook pra_restore_full.yaml \
-e "pra_restore_dump=/var/backups/atlas/db-remote/glpi_dump_20260715_195001.sql.gz"
```

Pourquoi cette étape ?

Sans `pra_restore_dump`, le playbook prend *automatiquement* le dump le plus récent sur `atlbp01p` (`ls -t | head -1`) : on repart du point de récupération le plus proche (RPO minimal). On ne fournit un dump précis que pour rejouer un état antérieur.

Ce que fait le playbook, dans l'ordre.

1. Horodatage de début + journal `/var/log/atlas/pra_restore_full.log`.
2. Reprovisioning LXC, socle common, bastion.
3. Services : MariaDB → SMTP → GLPI → reverse proxy → Zabbix → backup.
4. Données (bloc `block/rescue/always`) : transfert du dump `atlbp01p` → `atldb01p`, import dans `glpi_db`, restauration du filestore, puis **suppression du dump temporaire**.
5. Vérifications : GLPI en HTTP direct *et* via le reverse proxy HTTPS, comptage des tickets, port Zabbix 10051 ouvert.
6. Calcul et journalisation du **RTO mesuré** (comparé à la cible 40 min).

Attention

En cas d'échec pendant la restauration des données, le bloc `rescue` journalise l'**ÉCHEC** et `any_errors_fatal` interrompt le PRA : jamais de poursuite sur une base à moitié restaurée. Le `always` nettoie systématiquement le dump temporaire.

Contrôle final. Rouvrir GLPI (Étape 1.a) et vérifier que les tickets sont bien là : la boucle « je perds tout → je restaure → le service revient » est bouclée.

8. Étape 6 — Reprise ciblée : restauration granulaire

Cas plus fin : des tickets ont été supprimés, mais l'infra est saine. On ne restaure **QUE** les tickets manquants, sans écraser ceux créés depuis.

```
# 1) Choisir le dump anterieur a l'incident (sur atldb01p)
ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'

# 2) Lancer la restauration granulaire (dump OBLIGATOIRE)
ansible-playbook pra_restore_granular.yaml \
-e "pra_target_dump=/var/backups/atlas/db/glpi_dump_20260715_193001.sql.gz"
```


Pourquoi cette étape ?

`pra_target_dump` est **obligatoire** (un **assert** arrête le playbook si elle manque) : restaurer des tickets est chirurgical, on choisit explicitement la photo à réinjecter. Aucun automatisme ici, contrairement à l'Étape 5.

Mécanisme de sûreté. Le dump est restauré dans une base *temporaire* (`glpi_restore_tmp`), jamais en production. Seules les tables de tickets sont exportées avec `mysqldump -no-create-info -insert-ignore`, puis fusionnées : les `INSERT IGNORE` ne touchent pas les lignes déjà présentes. **Les tickets créés après l'incident ne sont donc jamais écrasés.** La base temporaire est supprimée en fin de course et le RPO effectif (âge du dump) est journalisé.

9. Aide-mémoire — commandes essentielles

Étape	Objectif	Commande
1	Accéder à GLPI	<code>ssh -L 8443:10.20.0.10:443 root@...</code>
1	Accéder à Zabbix	<code>ssh -L 8080:10.30.0.12:8080 root@...</code>
1	Accéder à Grafana	<code>ssh -L 3000:10.30.0.14:3000 root@...</code>
2	Vérifier la syntaxe	<code>ansible-playbook site.yaml --syntax-check</code>
2	Simuler (dry-run)	<code>ansible-playbook site.yaml --check --diff</code>
2	Déployer / cibler	<code>ansible-playbook site.yaml [-tags X -limit Y]</code>
3	Provoquer un incident	<code>ansible atl1st01p -m command -a ".../chaos.sh run"</code>
3	Réparer	<code>ansible atl1st01p -m command -a ".../chaos.sh clean"</code>
4	Vérifier les backups	<code>ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/'</code>
5	Restauration complète	<code>ansible-playbook pra_restore_full.yaml</code>
6	Restauration ciblée	<code>ansible-playbook pra_restore_granular.yaml -e "pra_target_dump=..."</code>

TABLE 4 – Récapitulatif opérationnel, dans l'ordre chronologique du déroulé.

10. Annexe — Corrections intégrées à l'IaC (juillet 2026)

Trois corrections de supervision ont été appliquées puis **pérennisées dans le playbook** (elles survivent donc à toute réexécution) :

Fix	Correction	Fichier
B	Flux firewall supervision → bastion (VLAN30→90, 10050/10051)	roles/proxmox_network/defaults/main.yaml
C	Agent Zabbix accepte les checks passifs locaux (127.0.0.1)	roles/zabbix_agent/templates/zabbix_agent2.conf.j2
A	Désactivation des items internes « not supported » (bruit)	roles/zabbix/tasks/tune_health.yaml

TABLE 5 – Un `-tags proxmox_network,zabbix` redéploie l'ensemble.